

Data Quality and Validation in ETL

Question 1 : Define Data Quality in the context of ETL pipelines. Why is it more than just data cleaning?

Data Quality in ETL refers to ensuring that data is accurate, complete, consistent, valid, unique, and timely throughout the extract, transform, and load process. It is more than data cleaning because it not only fixes incorrect or missing data but also includes validating data against business rules, preventing errors, monitoring quality metrics, and maintaining reliable data throughout the entire ETL pipeline.

Question 2 : Explain why poor data quality leads to misleading dashboards and incorrect decisions.

Poor data quality leads to misleading dashboards because inaccurate, incomplete, duplicate, or inconsistent data produces incorrect reports and KPIs. Decision-makers rely on these dashboards, so poor data quality can result in wrong forecasts, poor planning, financial losses, and reduced trust in business reports. Therefore, ensuring data quality in ETL pipelines is critical for accurate analytics and informed decision-making.

Question 3 : What is duplicate data? Explain three causes in ETL pipelines.

Duplicate data is the presence of multiple records for the same entity in a database. In ETL pipelines, it is mainly caused by multiple data sources, repeated ETL loads, and missing or incorrect primary keys. Duplicate data leads to inaccurate reports, incorrect analytics, and poor business decisions, so ETL processes should include proper validation and deduplication techniques.

Question 4 : Differentiate between exact, partial, and fuzzy duplicates.

There are three types of duplicate data. Exact duplicates are completely identical records. Partial duplicates have some matching fields but differ in others, such as phone number or address. Fuzzy duplicates refer to the same entity but contain spelling mistakes, abbreviations, or formatting differences. While exact duplicates are easy to detect, fuzzy duplicates require advanced matching techniques.

Question 5 : Why should data validation be performed during transformation rather than after loading?

Data validation should be performed during the Transformation stage because it detects and fixes errors before the data is loaded into the target system. This prevents invalid data from entering the database, improves data quality, reduces the cost of corrections, and ensures accurate reports and reliable business decisions.

Question 6 : Explain how business rules help in validating data accuracy. Give an example.

Business rules validate data by checking whether it meets predefined business requirements before loading. They help maintain accurate, consistent, and reliable data, ensuring trustworthy analytics and decision-making.

For example, in the next given dataset, Txn_ID 206 has a NULL transaction amount and Customer_Name as 'N/A', which violates business rules. Such records should be rejected or corrected before loading to maintain high-quality data.

Question 7 : Write an SQL query on to list all duplicate keys and their counts using the business key (Customer_ID + Product_ID + Txn_Date + Txn_Amount)

```
SELECT
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount,
    COUNT(*) AS Duplicate_Count
FROM Sales_Transactions
GROUP BY
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount
HAVING COUNT(*) > 1;
```

To find duplicate business keys, we group the data by the business key columns—Customer_ID, Product_ID, Txn_Date, and Txn_Amount—and use COUNT(*) to count occurrences. Using HAVING COUNT(*) > 1, we return only those combinations that appear more than once. In the given dataset, the combination (C101, P11, 2025-12-01, 4000) appears 3 times, so it is identified as a duplicate.

Question 8 :Identify Sales_Transactions.Customer_ID values that violate referential integrity when joined with Customers_Master and write a query to detect such violations.

Referential integrity ensures that every foreign key value exists in the parent table. Here, Customer_ID in Sales_Transactions is a foreign key referencing CustomerID in Customers_Master. Using a LEFT JOIN and checking for NULL values, we can identify invalid customer IDs. In the given dataset, C105 and C106 do not exist in the master table, so they violate referential integrity.

```
SELECT s.Txn_ID, s.Customer_ID  
FROM Sales_Transactions s  
LEFT JOIN Customers_Master c  
ON s.Customer_ID = c.CustomerID  
WHERE c.CustomerID IS NULL;
```